



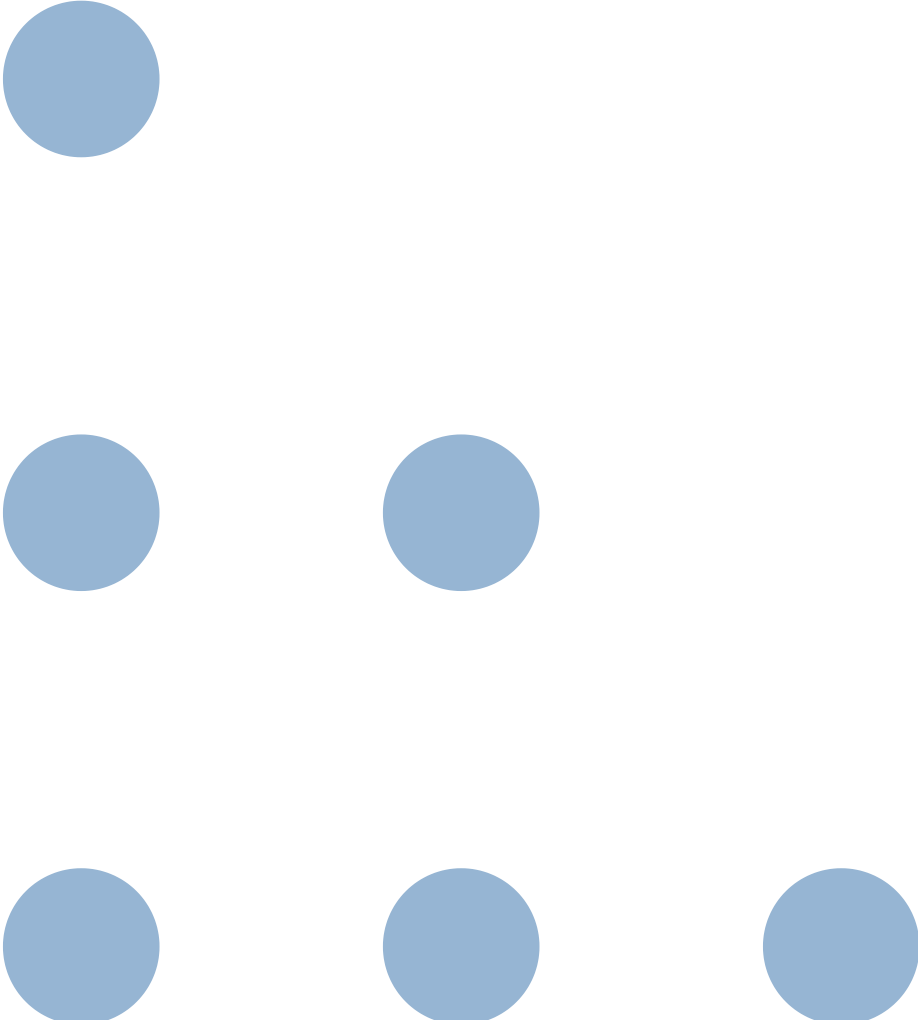
ACCESO A DATOS

UT2. FICHEROS

POR ALICIA CÁMARA



RESUMEN DE CONTENIDOS



INTRODUCCIÓN

OPERACIONES CON ARCHIVOS

ARCHIVOS DE TEXTO

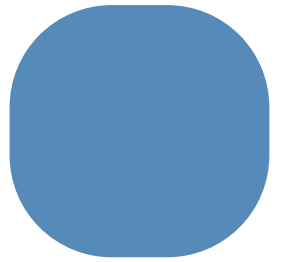
ARCHIVOS TEXTO PLANO

ARCHIVOS PROPERTIES

ARCHIVOS XML

ARCHIVOS BINARIOS

ACCESO ALEATORIO



UT2

INTRODUCCIÓN

1. INTRODUCCIÓN

En Java, el manejo de ficheros es una parte esencial para interactuar con el sistema de archivos y realizar operaciones de entrada y salida.

La clase fundamental para estas tareas es **nio.file**, que proporciona una interfaz para representar tanto archivos como directorios. A través de File, los programadores pueden realizar operaciones como la **creación y eliminación de archivos, navegación por directorios, comprobación de existencia, y obtención de información sobre archivos**.

El manejo de excepciones con **IOException** , es crucial al trabajar con ficheros para garantizar una ejecución segura y controlada. Este aspecto es esencial para el desarrollo de aplicaciones que requieren interacción con archivos y directorios en el entorno Java.

En este tema veremos el manejo de diferentes tipos de archivos, así como su uso en la implementación de persistencia de datos en java.

1. INTRODUCCIÓN

TIPOS DE ARCHIVOS

Los **archivos de texto** y los **archivos binarios** son dos tipos fundamentales de archivos que se diferencian principalmente en cómo almacenan los datos y cómo estos son interpretados.



Un **archivo de texto** contiene datos que están representados en un **formato legible por humanos**. Esto significa que los datos en un archivo de texto son una secuencia de caracteres codificados (normalmente en conjuntos como ASCII, UTF-8, o UTF-16), donde cada byte o grupo de bytes representa un carácter (letras, números, símbolos, espacios, saltos de línea, etc.).

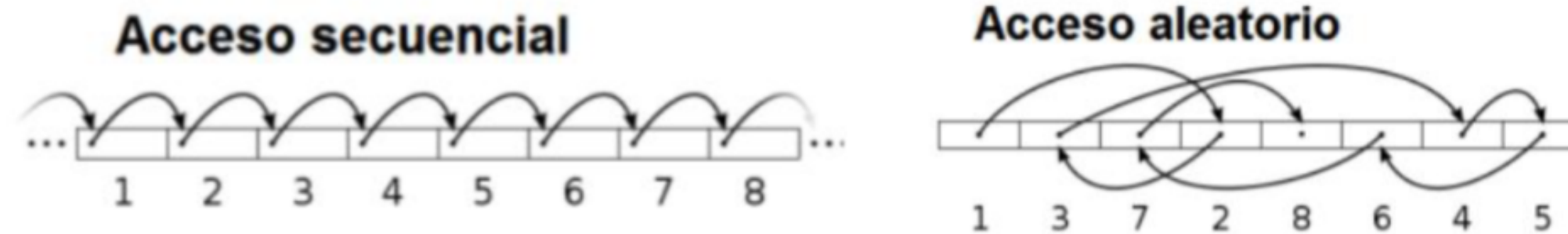
Un **archivo binario** contiene datos que están representados directamente en formato binario (**secuencias de bytes**), y no están destinados a ser leídos directamente por humanos. Los archivos binarios pueden contener cualquier tipo de datos: números, imágenes, audio, vídeo, programas, etc. A diferencia de los archivos de texto, los archivos binarios no siguen una codificación de caracteres específica.



1. INTRODUCCIÓN

FORMAS DE ACCESO A UN FICHERO

Existen dos formas principales de acceder a un fichero: **acceso secuencial** y **acceso aleatorio**. Ambas se utilizan según las necesidades de lectura y escritura de los datos en el archivo.



El **acceso secuencial** es la forma más simple de interactuar con archivos, leyendo o escribiendo datos de manera ordenada desde el principio hasta el final. No permite saltar a posiciones específicas, lo que lo hace ineficiente para búsquedas rápidas. Es común en archivos de texto donde los datos están estructurados en secuencias, como logs o CSV.

El **acceso aleatorio** permite leer y escribir datos en cualquier posición del archivo sin tener que procesarlo de forma secuencial. Esto significa que **puedes "saltar" a cualquier punto del archivo** y realizar operaciones en esa ubicación específica. Es útil para grandes volúmenes de datos o cuando hay que acceder rápidamente a posiciones específicas dentro del archivo. Por ejemplo, los archivos de vídeo o audio suelen utilizar acceso aleatorio para saltar rápidamente a una posición de tiempo específica.

1. INTRODUCCIÓN

CLASES JAVA PARA EL ACCESO A FICHEROS

ACCESO SECUENCIAL		
TIPO FICHERO	LECTURA	ESCRITURA
Texto	FileReader , generalmente combinado con BufferedReader para mejorar la eficiencia	FileWriter , combinado con BufferedWriter
Texto	Files.readAllLines() - Paquete java.nio2	Files.write(Path, byte[]) - Paquete java.nio2
Binario	ObjectInputStream	ObjectOutputStream
ACCESO ALEATORIO		
Binario	RandomAccessFile	RandomAccessFile

FileReader (versión antigua I/O) es más eficiente para archivos grandes ya que permite cargar en memoria línea a línea; mientras que Files.readAllLines() es más moderno, conveniente y fácil de usar, pero puede consumir más memoria si el archivo es grande, ya que lee todo el archivo en una lista.

1. INTRODUCCIÓN

CLASE SYSTEM PROPERTIES

Para el acceso a ficheros, independientemente del tipo de los mismos, conviene conocer el funcionamiento de las System Properties de las que dispone Java en su API. Estas permiten acceder a propiedades de la configuración del sistema y, entre otras, podemos encontrarnos con algunas muy interesantes relacionadas con este tema:

- **file.separator**: Obtiene el caracter, según el Sistema Operativo, para la separación de las rutas (/ ó \). También se puede utilizar la constante **File.separator**
- **user.home**: Obtiene la ruta de la carpeta personal del usuario, que dependerá del Sistema Operativo en cada caso
- **user.dir**: Obtiene la ruta en la que se encuentra actualmente el usuario
- **line.separator**: Obtiene el carácter que separa las líneas de un fichero de texto (difiere entre Windows/Linux)

En el caso de que se quiera acceder al valor de alguna de estas propiedades debe hacerse utilizando la llamada al método **System.getProperty(String)**

1. INTRODUCCIÓN

CLASE SYSTEM PROPERTIES

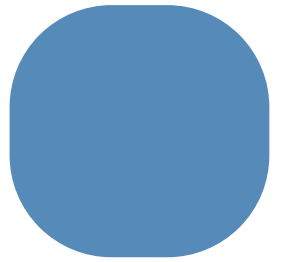
Ejemplo de uso de las System Properties:

```
public final static String SEPARATOR_CHAR = System.getProperty("file.separator");

public static void main(String[] args) {
    System.out.println("El separador del sistema operativo es: "
        + System.getProperty("file.separator"));
    System.out.println("La carpeta personal del usuario es: "
        + System.getProperty("user.home"));

    String rutaFichero = System.getProperty("user.home") + SEPARATOR_CHAR
        + "Downloads" + SEPARATOR_CHAR + "properties.txt";
    System.out.println("La ruta del fichero es: " + rutaFichero);
}
```

El separador del sistema operativo es: /
La carpeta personal del usuario es: [/Users/alicia](#)
La ruta del fichero es: /Users/alicia/Downloads/properties.txt



UT2

OPERACIONES CON ARCHIVOS

2. OPERACIONES CON ARCHIVOS

La API “java.nio.file” es una parte integral de la biblioteca de entrada/salida introducida en Java 7 para mejorar el manejo de archivos y directorios. Proporciona una interfaz más moderna y poderosa para operaciones de entrada/salida en comparación con la antigua clase “java.io.File”. Sus herramientas básicas son:

- Las clases Path y Paths.

La clase Path y Paths representan una ruta de sistema de archivos. Puedes obtener instancias de Path mediante métodos estáticos de la clase Paths. Esta ruta nos servirá para indicar el archivo o directorio con el que queremos trabajar.

```
// Creación de una instancia de ruta de archivo  
Path ruta = Paths.get("ruta/del/archivo.txt");
```

- La clase Files

La clase Files proporciona métodos estáticos para realizar operaciones comunes en archivos y directorios, como la lectura y escritura de archivos, comprobación de existencia, copia, eliminación, entre otros.

2. OPERACIONES CON ARCHIVOS

COPIAR

La clase Files contiene el método “**copy**” que copia el contenido de un archivo de origen a un archivo de destino.

Su funcionamiento básico es...

Files.copy(Path origen, Path destino)

```
Para copiar un archivo deberemos declarar las rutas origen y destino.  
Lo haremos con el objeto "Path".*/  
Path origenCopia = Paths.get("DataSet/Archivo.txt");  
Path destinoCopia = Paths.get("DataSet/CopiaArchivo.txt");  
  
try { // Todas las operaciones con ficheros deben ir en un bloque try-catch.  
  
    Files.copy(origenCopia, destinoCopia); // Posteriormente emplearemos el método estático "Files.copy"  
  
} catch (IOException e) {  
    // Si existe el archivo de destino dará error  
    System.err.println("ERROR en la copia del archivo");  
}
```

2. OPERACIONES CON ARCHIVOS

MOVER

La clase Files contiene el método “**move**” que mueve o renombra un archivo o directorio de origen a un destino.

Su funcionamiento básico es...

Files.move(Path origen, Path destino)

```
Para mover o renombrar un archivo deberemos declarar las rutas origen y destino.  
Lo haremos con el objeto "Path".*/  
Path origenArchivo = Paths.get("DataSet/Archivo.txt");  
Path destinoArchivo = Paths.get("DataSet/OtroDirectorio/Archivo.txt");  
  
try { // Todas las operaciones con ficheros deben ir en un bloque try-catch.  
    // Posteriormente emplearemos el método estático "Files.move"  
    // + Si los directorios de origen y destino son diferentes el archivo se moverá.  
    // + Si los directorios de origen y destino son iguales el archivo se renombrará.  
    Files.move(origenArchivo, destinoArchivo);  
} catch (IOException e) {  
    // Deberá existir el directorio de destino, si no dará error.  
    System.err.println("ERROR al mover el archivo");  
}
```

2. OPERACIONES CON ARCHIVOS

CREAR

Los métodos “**createFile**” y “**createDirectory**” que permiten crear archivos y directorios.

Su funcionamiento básico es...

Files.createFile(Path *ruta*)

Files.createDirectory(Path *ruta*)

```
// Quiero crear el directorio DataSet en mi ubicación.
// Luego quiero crear el archivo "Archivo.txt"
// Creo las rutas para trabajar
Path rutaDirectorio = Paths.get("DataSet");
Path rutaArchivo = Paths.get("DataSet/Archivo.txt");

// Todas la operaciones con ficheros deberán ir en un bloque try-catch.
try {
    if (Files.exists(rutaDirectorio)){ // Comprobamos si ya existe el directorio DataSet
        System.out.println("El directorio ya existe");
    }else{ // Si no existe el directorio DataSet lo creamos...
        Files.createDirectories(rutaDirectorio);
        System.out.println("Directorio creado en " + rutaDirectorio.toString());
        //Una vez creado el directorio crearemos el archivo
        Files.createFile(rutaArchivo);
    }
}catch (IOException e){
    e.printStackTrace();
}
}
```

2. OPERACIONES CON ARCHIVOS

BORRAR

La clase `Files` contiene los métodos “**delete**” y “**deleteIfExists**” que permiten borrar archivos y directorios.

Su funcionamiento básico es...

`Files.delete(Path ruta)`

`Files.deleteIfExists(Path ruta)`

```
// Quiero borrar el archivo "Archivo.txt" en el directorio DataSet.  
// Creo una ruta para trabajar  
Path ruta = Paths.get("DataSet/Archivo.txt");  
// Todas la operaciones con ficheros deberán ir en un bloque try-catch.  
try {  
    // Debemos comprobar la existencia del archivo porque, de lo contrario,  
    // generará una excepción.  
    Files.delete(ruta);  
  
    // Podemos usar el método "deleteIfExists" para comprobar primero su existencia.  
    // El método devolverá true o false dependiendo si se ha borrado el archivo o no.  
    Files.deleteIfExists(ruta);  
}  
catch (IOException e){  
    System.out.println("No existe el archivo");  
}
```

2. OPERACIONES CON ARCHIVOS

CONSULTAS

La clase `Files` contiene los métodos “**exists**”, “**isDirectory**” y “**isRegularFile**” que permiten consultar características de los archivos y directorios.

Su funcionamiento básico es...

Files.exists(Path *ruta*)

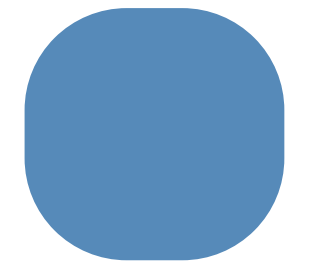
Verifica si el path especificado existe. Devuelve un valor boolean.

Files.isDirectory(Path *ruta*)

Verifica si el path especificado corresponde a un directorio. Devuelve un valor boolean.

Files.isRegularFile(Path *ruta*)

Verifica si el path especificado corresponde a un archivo. Devuelve un valor boolean.



UT2

ARCHIVOS DE TEXTO PLANO

3. ARCHIVOS DE TEXTO

TEXTO PLANO

La manipulación de archivos de texto es una tarea común en el desarrollo de software, y Java proporciona un conjunto robusto de clases y métodos para facilitar la lectura y escritura de datos en archivos.

Como hemos indicado al inicio, en este tema daremos una visión simplificada del manejo de archivos de texto, por lo que nos centraremos principalmente en el procesamiento de archivos de texto línea a línea.

Los métodos clásicos de lectura y escritura de archivos de texto en java involucraban múltiples clases del paquete `java.io`, pero para un manejo simple de archivos de texto es mucho más útil emplear la clase **Files** del paquete **java.nio**.

A continuación, se presentan las principales operaciones para **procesar ficheros de texto de forma secuencial** en Java.

- Leer todas las líneas de un archivo
- Escribir un archivo línea a línea
- Escribir un conjunto de líneas en un archivo

3. ARCHIVOS DE TEXTO

TEXTO PLANO - LECTURA

La mejor estrategia para procesar un archivo de texto es leer todas las líneas del archivo con **Files.readAllLines()** y guardarlas en una lista de Strings. De este modo podremos procesar cada línea por separado como una cadena de texto cualquiera. Su funcionamiento básico es...

```
// Quiero leer el archivo "Archivo.txt" del directorio DataSet.  
// Creo una ruta para trabajar  
Path ruta = Paths.get("DataSet/Archivo.txt");  
  
// Creamos una lista de String para guardar todas las líneas de texto del archivo  
// puede ser, por ejemplo un ArrayList.  
List<String> archivoDeTexto;  
  
// Todas la operaciones con ficheros deberán ir en un bloque try-catch.  
try {  
    if (Files.notExists(ruta)){ // Comprobamos si ya existe el archivo en DataSet  
        System.out.println("El archivo no existe");  
    }else{ // Si existe el archivo guardamos sus filas en la lista creada.  
        archivoDeTexto = Files.readAllLines(ruta);  
  
        // Ahora cada posición de la lista contiene una fila del archivo  
        // Podemos recorrer la lista para visualizarlo o procesar las cadenas como convenga.  
        for(String linea: archivoDeTexto){  
            System.out.println(linea);  
        }  
    }  
}catch (Exception e){  
    e.printStackTrace();  
}
```

3. ARCHIVOS DE TEXTO

TEXTO PLANO - ESCRITURA (I)

Del mismo modo que a la hora de leer, la mejor manera de escribir en un archivo de texto es línea a línea, para ello disponemos del método **Files.write()** y dos estrategias. La primera es escribir en el archivo una línea de cada vez.

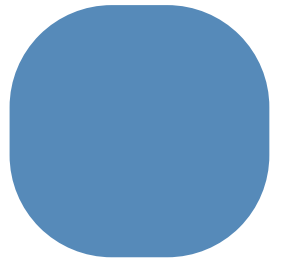
```
// Quiero escribir en el archivo "Archivo.txt" del directorio DataSet.  
// Creo una ruta para trabajar  
Path ruta = Paths.get("DataSet/Archivo.txt");  
  
// Todas la operaciones con ficheros deberán ir en un bloque try-catch.  
try {  
    if (Files.notExists(ruta)){ // Comprobamos si ya existe el archivo en DataSet  
        System.out.println("El archivo no existe");  
    }else{ // Si existe el archivo podemos escribir en él...  
  
        // Para escribir una línea en particular  
        String contenido = "Voy a escribir solo esta línea\n";  
        Files.write(ruta,contenido.getBytes()); // De este modo sobrescribo el contenido del fichero  
  
        // Para añadir una línea en vez de sobrescribir  
        contenido = "Voy a escribir otra línea\n";  
        Files.write(ruta,contenido.getBytes(), StandardOpenOption.APPEND);  
    }  
}catch (Exception e){  
    e.printStackTrace();  
}
```

3. ARCHIVOS DE TEXTO

TEXTO PLANO - ESCRITURA (II)

La otra estrategia es ir componiendo una lista con cada una de las líneas que queramos escribir en el archivo para, posteriormente, volcar la lista completa en el archivo.

```
// Quiero escribir en el archivo "Archivo.txt" del directorio DataSet.  
// Creo una ruta para trabajar  
Path ruta = Paths.get("DataSet/Archivo.txt");  
  
// Todas la operaciones con ficheros deberán ir en un bloque try-catch.  
try {  
    if (Files.notExists(ruta)){ // Comprobamos si ya existe el archivo en DataSet  
        System.out.println("El archivo no existe");  
    }else{ // Si existe el archivo podemos escribir en él...  
  
        // Para añadir un conjunto de líneas usaremos una lista de cadenas  
        List<String> lineas = new ArrayList<>();  
        lineas.add("Primera Línea");  
        lineas.add("Segunda Línea");  
        lineas.add("Tercera Línea");  
        lineas.add("...");  
  
        // De este modo guardará cada posición de la lista en una línea del archivo.  
        Files.write(ruta,lineas,StandardOpenOption.APPEND);  
    }  
}catch (Exception e){  
    e.printStackTrace();  
}
```



UT2

ARCHIVOS DE TEXTO: FICHEROS PROPERTIES

3. ARCHIVOS DE TEXTO

FICHEROS DE CONFIGURACIÓN

Un fichero de configuración es un archivo que **almacena parámetros y opciones** que una aplicación o sistema utiliza para funcionar correctamente. Estos archivos son utilizados para definir aspectos como la conexión a una base de datos, rutas de archivos, ajustes de red, variables de entorno, opciones de usuario, entre otros. Al **separar la configuración del código**, es más fácil mantener y actualizar los ajustes sin necesidad de recompilar o modificar el programa en sí.

Tradicionalmente, en Java se utilizan archivos con **extensión .properties** para almacenar la configuración de las aplicaciones. Estos archivos siguen un formato sencillo de **clave-valor**, y la clase **Properties** de Java está diseñada específicamente para manejar este tipo de ficheros

Sin embargo, no existe ninguna restricción que impida usar otros formatos como JSON o YAML. De hecho, en proyectos más modernos o complejos, JSON y YAML son opciones perfectamente válidas y se utilizan con frecuencia. La elección del formato dependerá de las necesidades y complejidad del proyecto, ya que cada uno tiene sus ventajas y desventajas.

3. ARCHIVOS DE TEXTO

FICHEROS DE CONFIGURACIÓN

Fichero .properties

VENTAJAS

- Formato simple y nativo en Java
- La clase Properties está diseñada específicamente para este formato
- Adecuado para configuraciones sencillas

DESVENTAJAS

- Limitado para configuraciones más complejas (jerarquías anidadas o estructuras complejas).

```
ejemplo.properties
# Fichero de configuracion
username=admin
password=1234
port=8006
server.url=http://example.com
```

Es el que vamos a estudiar

3. ARCHIVOS DE TEXTO

FICHEROS DE CONFIGURACIÓN

Fichero .json

VENTAJAS

- Usado en muchos lenguajes modernos, incluyendo JavaScript, Python, Node.js, etc.
- Formato ampliamente adoptado y soporta estructuras más complejas (anidamientos y listas).

DESVENTAJAS

- Java no tiene una clase nativa para trabajar con archivos JSON. Hay que usar bibliotecas externas como Jackson o Gson.
- Más complicado de manejar que .properties para configuraciones simples.

```
config.json
{
  "server": {
    "host": "localhost",
    "port": 8080
  },
  "database": {
    "user": "admin",
    "password": "admin123"
  }
}
```

3. ARCHIVOS DE TEXTO

FICHEROS DE CONFIGURACIÓN

Fichero .yaml

VENTAJAS

- Usado en aplicaciones modernas como Docker y Kubernetes.
- Es legible y soporta estructuras jerárquicas y complejas, similar a JSON, pero con una sintaxis más simple y menos verbosa.

DESVENTAJAS

- No hay soporte nativo en Java, por lo que necesitas una biblioteca externa como SnakeYAML.
- La sintaxis de YAML puede ser más propensa a errores por el uso de espacios y tabulaciones.

```
config.yaml
server:
  host: localhost
  port: 8080
database:
  user: admin
  password: admin123
```

3. ARCHIVOS DE TEXTO

FICHEROS DE CONFIGURACIÓN - LECTURA

A la hora de leer los ficheros de propiedades, en vez de tener que recorrer todo el fichero como suele ocurrir con los ficheros de texto, simplemente tendremos que cargarlo (**load(FileInputStream)**) e indicar de qué propiedad queremos obtener su valor (**getProperty(String)**).

¡CUIDADO! Al tratarse de ficheros de texto, toda la información se almacena como si de un String se tratara. Para trabajar con otro tipo de datos será necesario realizar las conversiones oportunas.



```
private final static String RUTA_FICHERO_CONF = "resources/config.properties";

public static void main(String[] args) {
    // Creamos un objeto Properties
    Properties properties = new Properties();

    // Capturamos la excepción que puede provocar FileInputStream
    try {
        // Cargamos el archivo .properties
        properties.load(new FileInputStream(RUTA_FICHERO_CONF));

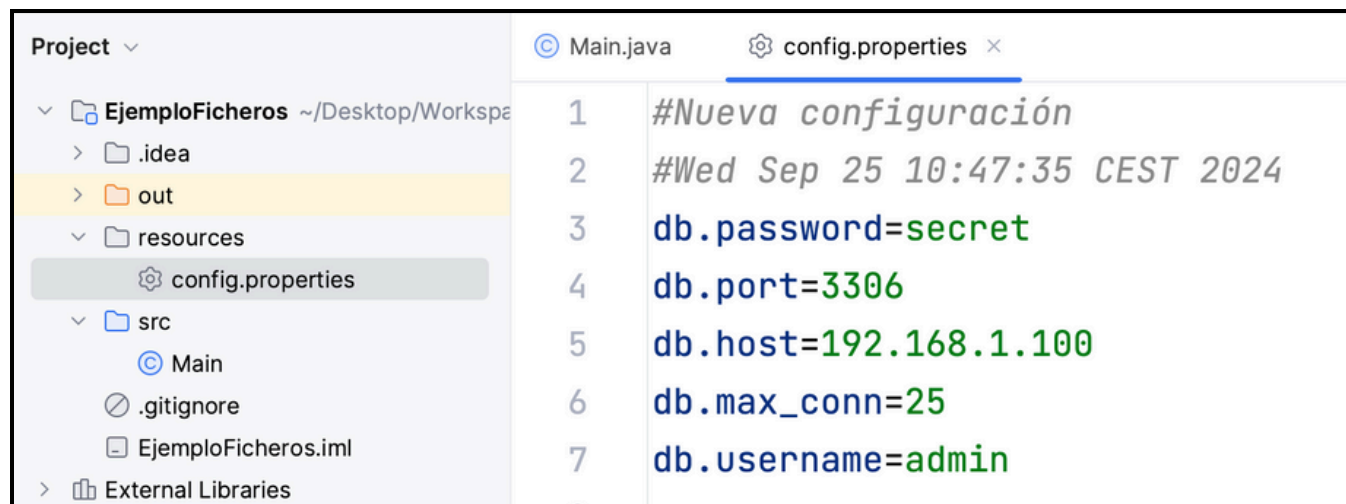
        // Recuperamos las propiedades que necesitamos
        String host = properties.getProperty("db.host");
        String passwd = properties.getProperty("db.password");
        int maxConnections = Integer.parseInt(properties.getProperty("db.max_conn"));

    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

3. ARCHIVOS DE TEXTO

FICHEROS DE CONFIGURACIÓN - ESCRITURA

Para modificar el valor de una propiedad, cargaremos el fichero con el método **load(new FileInputStream)** y modificaremos las propiedades con **setProperty(String key, String value)**. Cuando queramos guardar el contenido, ejecutaremos **store(OutputStream, String comment)**.



```
private final static String RUTA_FICHERO_CONF = "resources/config.properties";

public static void main(String[] args) {
    // Creamos un objeto Properties
    Properties properties = new Properties();

    // Capturamos la excepción que puede provocar FileInputStream
    try {
        // Cargamos el archivo .properties
        properties.load(new FileInputStream(RUTA_FICHERO_CONF));

        // Modificamos las propiedades
        properties.setProperty("db.host", "192.168.1.100");

        // Queremos incrementar en 5 el número máximo de conexiones
        int maxConnections = Integer.parseInt(properties.getProperty("db.max_conn"));
        maxConnections = maxConnections + 5;
        properties.setProperty("db.max_conn", String.valueOf(maxConnections));

        properties.store(new FileOutputStream(RUTA_FICHERO_CONF), comments: "Nueva configuración");
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```



UT2

ARCHIVOS DE TEXTO: FICHEROS XML

3. ARCHIVOS DE TEXTO

FICHEROS XML

Un **fichero XML** (eXtensible Markup Language) es un formato de archivo de texto que se utiliza para **almacenar y transportar datos de manera estructurada y legible**. XML es un lenguaje de marcado que permite a los usuarios definir sus propias etiquetas y estructuras de datos, lo que lo convierte en un formato muy flexible y ampliamente utilizado para representar información jerárquica.

Java proporciona varias librerías para trabajar con XML, siendo las más comunes:

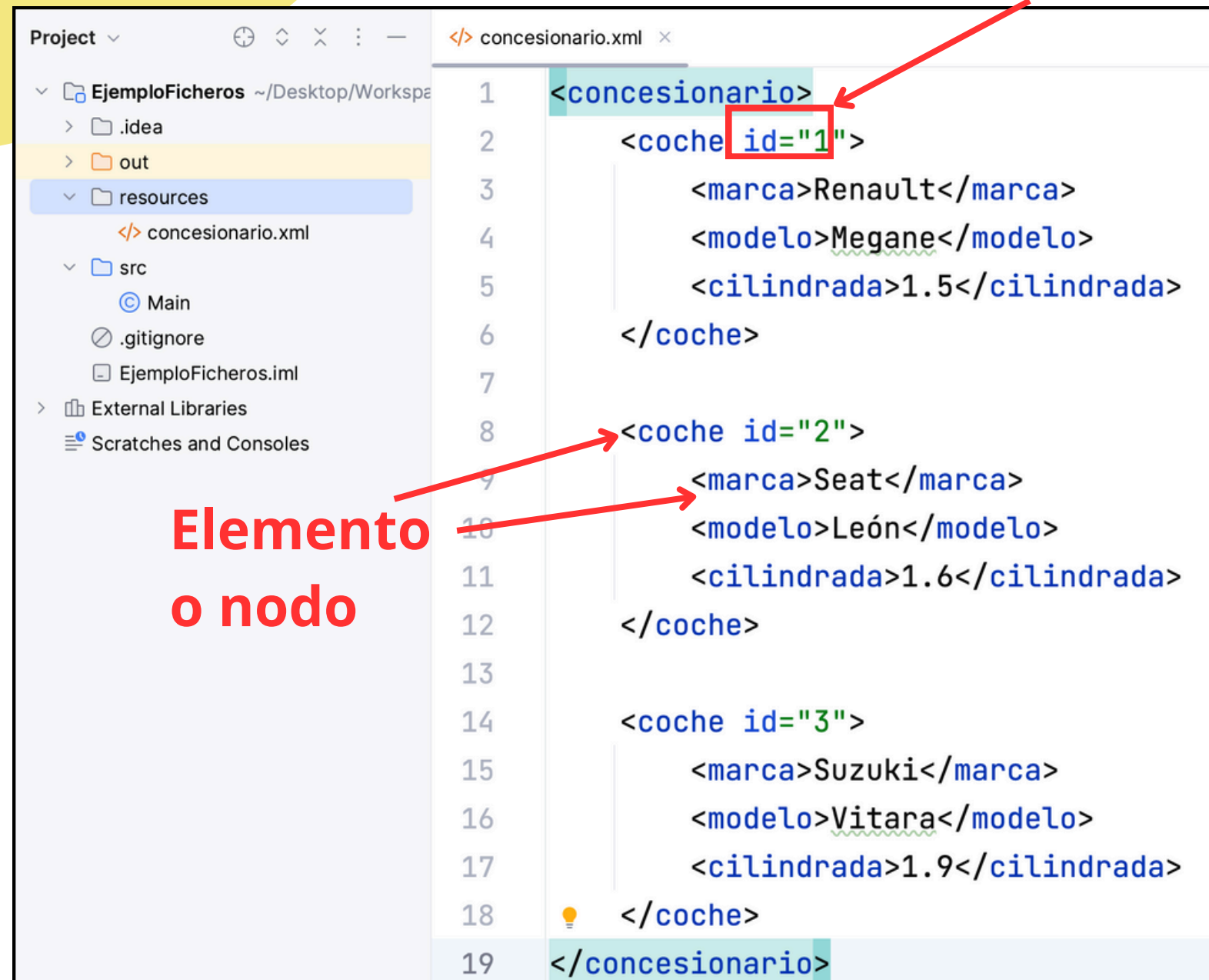
1. **DOM (Document Object Model)**: Carga el documento XML completo en memoria y genera un árbol de objetos llamados nodos que representan cada etiqueta del documento XML. Estos nodos pueden ser de muchos tipos: documentos, elementos, atributos, etc., que se relacionan entre sí mediante relaciones padre-hijo. Esta es la que utilizaremos nosotros.
2. **SAX (Simple API for XML)**: Lee el documento XML de manera secuencial y se utiliza principalmente para análisis, no para modificación.
3. **StAX (Streaming API for XML)**: Proporciona una API basada en eventos para procesar XML de manera más eficiente que DOM y permite tanto lectura como escritura.

3. ARCHIVOS DE TEXTO

FICHEROS XML - LECTURA

Supongamos que tenemos el siguiente fichero XML:

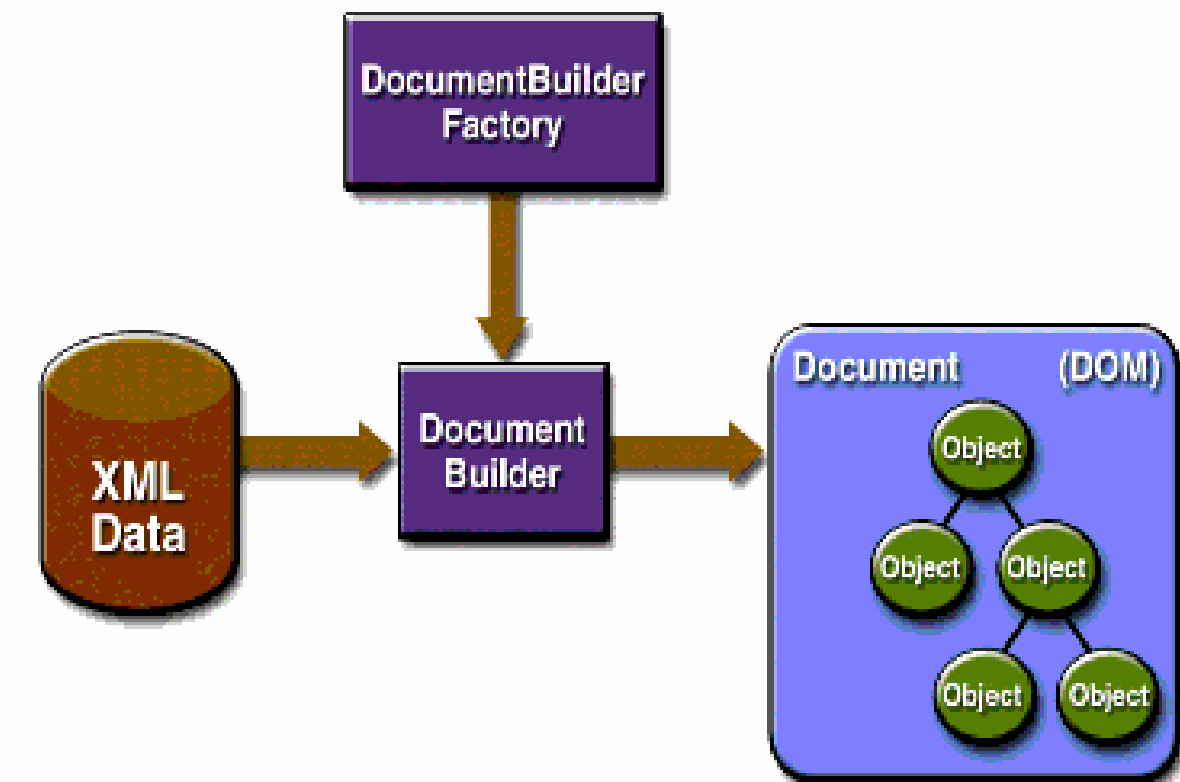
Atributo



```
1 <concesionario>
2   <coche id="1">
3     <marca>Renault</marca>
4     <modelo>Megane</modelo>
5     <cilindrada>1.5</cilindrada>
6   </coche>
7
8   <coche id="2">
9     <marca>Seat</marca>
10    <modelo>León</modelo>
11    <cilindrada>1.6</cilindrada>
12  </coche>
13
14  <coche id="3">
15    <marca>Suzuki</marca>
16    <modelo>Vitara</modelo>
17    <cilindrada>1.9</cilindrada>
18  </coche>
19 </concesionario>
```

**Elemento
o nodo**

A la hora de leer un archivo XML a través de DOM debemos instanciar una serie de clases antes de poder tratar el fichero



3. ARCHIVOS DE TEXTO

FICHEROS XML - LECTURA

Tras instanciar las clases necesarias (DocumentBuilderFactory y DocumentBuilder), debemos acceder al elemento raíz del fichero con el método **getDocumentElement()**, en este caso <concesionario>, y normalizarlo con el método **normalize()**. La normalización facilita el tratamiento del fichero ya que elimina los nodos vacíos y combina adyacentes en caso de que los hubiera.

```
private final static String RUTA_FICHERO_CONF = "resources/concesionario.xml";

public static void main(String[] args) {
    try {
        // Declaramos las clases necesarias para DOM
        DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
        DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
        Document doc = dBuilder.parse(new File(RUTA_FICHERO_CONF));

        // Normalizamos el archivo XML
        doc.getDocumentElement().normalize();

        // Aquí va el código para procesar los elementos

    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

3. ARCHIVOS DE TEXTO

FICHEROS XML - LECTURA

El método

getElementsByTagName(String tag)

devuelve un objeto **NodeList**, similar a un **ArrayList**, que contiene todos los elementos del documento que coinciden con la etiqueta especificada como parámetro. Es importante tener en cuenta que tanto `<coche>` como `<marca>` son elementos XML, pero se procesan de manera diferente debido a sus características individuales: `<coche>` es un elemento complejo que contiene otros elementos, mientras que `<marca>` es un elemento simple que únicamente contiene texto. Aunque ambos se recuperan utilizando el método `getElementsByTagName(String)`, su procesamiento difiere.

```
System.out.println("El elemento raíz es: " + doc.getDocumentElement().getTagName());

// Obtenemos todos los elementos "coche" (elemento complejo)
NodeList listaCoches = doc.getElementsByTagName("coche");
System.out.println("Número de coches: " + listaCoches.getLength());

// Procesamos cada uno de los coches
for (int i = 0; i < listaCoches.getLength(); i++) {
    // Recuperamos el nodo en la posición i
    Node nodoCoche = listaCoches.item(i);
    nodoCoche.getAttributes();

    // Nos aseguramos que el nodo es un elemento (y no es, por ejemplo, un comentario)
    if (nodoCoche.getNodeType() == Node.ELEMENT_NODE) {
        // Convertimos el nodo a Element para poder procesarlo
        Element coche = (Element) nodoCoche;

        // Recuperamos el atributo id
        String id = coche.getAttribute(name: "id");

        // Recuperamos los elementos simples
        String marca = coche.getElementsByTagName("marca").item(index: 0).getTextContent();
        String modelo = coche.getElementsByTagName("modelo").item(index: 0).getTextContent();
        double cilindrada = Double.parseDouble(coche.getElementsByTagName("cilindrada").item(index: 0).getTextContent());

        System.out.println("Coche " + id + ". Marca: " + marca + ". Modelo: " + modelo + ". Cilindrada: " + cilindrada);
    }
}
```

3. ARCHIVOS DE TEXTO

FICHEROS XML - ESCRITURA

AÑADIR ELEMENTOS A XML YA EXISTENTE

Vamos a suponer que ha llegado un coche nuevo al concesionario y quieren añadirlo a su base de datos XML.

PRIMER PASO

Creamos el elemento coche (**createElement(String tag)**) con su atributo id (**element.setAttribute(nombre, value)**).

Posteriormente, crearemos un elemento por cada etiqueta hija (marca, modelo...) con su contenido textual y añadimos cada uno de esos hijos al coche (**element.appendChild(Element hijo)**).

```
// ... Inicializamos DocumentBuilderFactory, etc.

// Creamos el elemento coche
Element cocheNuevo = doc.createElement( tagName: "coche");
cocheNuevo.setAttribute( name: "id", value: "4");

// Creamos el elemento marca
Element marca = doc.createElement( tagName: "marca");
// Le añadimos el valor como nodo de texto
marca.appendChild(doc.createTextNode( data: "Toyota"));
// Incluimos la marca como elemento hijo de coche
cocheNuevo.appendChild(marca);

// Mismo proceso para el resto de hijos de coche
Element modelo = doc.createElement( tagName: "modelo");
modelo.appendChild(doc.createTextNode( data: "Corola"));
cocheNuevo.appendChild(modelo);

Element cilindrada = doc.createElement( tagName: "cilindrada");
cilindrada.appendChild(doc.createTextNode( data: "1.8"));
cocheNuevo.appendChild(cilindrada);
```

3. ARCHIVOS DE TEXTO

FICHEROS XML - ESCRITURA

AÑADIR ELEMENTOS A XML YA EXISTENTE

SEGUNDO PASO

Añadimos el coche creado al elemento concesionario que ya existe en el documento XML.

TERCER PASO

Guardamos los cambios realizados sobre el fichero XML mediante la clase Transformer

```
// Añadimos el nuevo coche como hijo del elemento raíz concesionario
// Recuperamos todos los nodos con nombre concesionario
NodeList concesionarioList = doc.getElementsByTagName("concesionario");
// Sabemos que solo hay un nodo concesionario por lo que recuperamos el elemento
Element concesionario = (Element) concesionarioList.item(index: 0);
concesionario.appendChild(cocheNuevo);

// Guardar los cambios en el fichero XML
TransformerFactory transformerFactory = TransformerFactory.newInstance();
Transformer transformer = transformerFactory.newTransformer();
DOMSource source = new DOMSource(doc);
StreamResult result = new StreamResult(new File(RUTA_FICHERO_CONF));
transformer.transform(source, result);

System.out.println("Se ha añadido un coche");
```

3. ARCHIVOS DE TEXTO

FICHEROS XML - ESCRITURA

CREAR EL FICHERO XML

Este caso es similar al anterior, con algunas modificaciones:

- Al inicializar el documento, debemos partir de uno nuevo (**dBuilder.newDocument()**) en vez de parsear uno ya existente
- Debemos crear también el elemento raíz y todos los hijos que queremos que dependan de él, así como la información de dichos hijos.

```
// Declaramos las clases necesarias para DOM
DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
Document doc = dBuilder.newDocument();
```

```
// Definimos el elemento raíz
Element root = doc.createElement( tagName: "concesionario");
doc.appendChild(root);
```

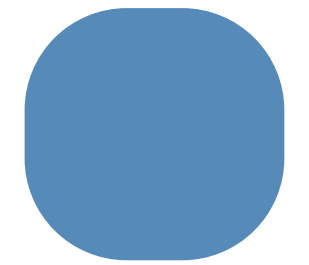
```
// Definimos el nodo que contiene los valores del coche
Element coche = doc.createElement( tagName: "coche");
coche.setAttribute( name: "id", value: "1");
root.appendChild(coche);
```

```
// Definimos cada uno de los hijos de coche y le asignamos valor
Element marca = doc.createElement( tagName: "marca");
marca.appendChild(doc.createTextNode( data: "Renault"));
coche.appendChild(marca);
```

```
Element modelo = doc.createElement( tagName: "modelo");
modelo.appendChild(doc.createTextNode( data: "Megan"));
coche.appendChild(modelo);
```

```
Element cilindrada = doc.createElement( tagName: "cilindrada");
cilindrada.appendChild(doc.createTextNode( data: "1.5"));
coche.appendChild(cilindrada);
```

```
// TODO: Confirmamos los cambios en el archivo XML mediante Transformer...
```



UT2

ARCHIVOS BINARIOS

4. ARCHIVOS BINARIOS

Java también nos permite procesar archivos binarios. La manipulación de archivos binarios en Java se realiza de manera similar a la manipulación de archivos de texto, pero en lugar de trabajar con caracteres, se tratan **bytes**.

La principal utilidad de los archivos binarios en java es garantizar la **persistencia de objetos**. Esta persistencia se implementa mediante la **serialización** de los mismos.

La serialización de objetos en Java es un mecanismo que permite convertir un objeto y su estado en una secuencia de bytes, que luego puede ser almacenada en un archivo, enviada a través de una red o almacenada en una base de datos. Este proceso permite persistir la información de un objeto y luego deserializarlo para reconstruir el objeto original. Java proporciona la interfaz "**Serializable**" y las clases "**ObjectOutputStream**" y "**ObjectInputStream**" para realizar la serialización y deserialización.



4. ARCHIVOS BINARIOS

GUARDAR OBJETOS

Para grabar un objeto emplearemos las clases “**Files**” de java.nio y “**OutputStream**” - “**ObjectOutputStream**” de java.io. Lo haremos de la siguiente manera.

Es importante recordar que aquellos objetos que quieras serializar deben implementar la interfaz “**Serializable**”. Si dentro de la clase hay atributos que son otras clases, éstos a su vez también deben ser Serializable. En este ejemplo, la clase persona implementa la interfaz.

```
public class Persona implements Serializable {  
    private String Nombre;  
    private int edad;  
    private double altura;  
  
    public Persona(String nombre, int edad, double altura) {  
        Nombre = nombre;  
        this.edad = edad;  
        this.altura = altura;  
    }  
}
```

```
// Creamos la lista de personas  
// Los objetos que queramos serializar deben implementar la interfaz "Serializable"  
List<Persona> listaDePersonas= new ArrayList<>();  
listaDePersonas.add(new Persona("Juan", 18, 1.78));  
listaDePersonas.add(new Persona("Ana", 25, 1.90));  
listaDePersonas.add(new Persona("Luisa", 21, 1.66));  
  
// Creamos una ruta para guardar el objeto  
Path ruta = Paths.get("DataSet/listaDePersonas.bin");  
  
// Creamos un flujo de bits de salida para serialización con ...  
try {  
    // Asignamos un archivo a un flujo de salida de datos  
    OutputStream archivoSalida = Files.newOutputStream(ruta);  
    // Especificamos que se trata de un flujo de salida de objetos  
    ObjectOutputStream flujoSalida = new ObjectOutputStream(archivoSalida);  
    // Escribimos el objeto en el flujo de salida.  
    flujoSalida.writeObject(listaDePersonas);  
}  
catch (Exception e){  
    e.printStackTrace();  
}
```

4. ARCHIVOS BINARIOS

RECUPERAR OBJETOS

Para recuperar un objeto emplearemos las clases “**Files**” de java.nio y “**InputStream**” - “**ObjectInputStream**” de java.io. Lo haremos de la siguiente manera.

```
// Creamos la lista de personas para guardar el objeto que leamos del archivo
List<Persona> listaDePersonas;

// Creamos una ruta para leer el archivo
Path ruta = Paths.get("DataSet/listaDePersonas.bin");

try {
    // Asignamos el archivo de lectura a un flujo de entrada de datos
    InputStream ArchivoEntrada = Files.newInputStream(ruta);
    // Especificamos que se trata de un flujo de entrada de objetos
    ObjectInputStream flujoEntrada = new ObjectInputStream(ArchivoEntrada);
    // Leemos el flujo de entrada y lo asignamos a la lista, especificando el tipo de objeto que estamos leyendo.
    listaDePersonas = (List<Persona>) flujoEntrada.readObject();

    // Comprobamos la correcta recuperación del objeto.
    for (Persona p:listaDePersonas){
        System.out.println(p);
    }
}catch (Exception e){
    e.printStackTrace();
}
```

4. ARCHIVOS BINARIOS

ATRIBUTO SERIAL VERSION

Cuando haces que una clase sea serializable, el compilador te puede sugerir que definas un campo de tipo long llamado **serialVersionUID**. Este atributo se utiliza para mantener la compatibilidad de la serialización entre diferentes versiones de una clase.

Cuando serializas un objeto, la estructura interna de la clase también se serializa junto con los datos. Al deserializar el objeto, Java verifica que la versión de la clase actual (en tiempo de deserialización) sea compatible con la versión de la clase que fue usada al momento de la serialización.

Si no defines explícitamente un serialVersionUID, el compilador lo generará automáticamente basado en los detalles de la clase (nombres de métodos, campos, interfaces, etc.). Esto es problemático porque si cambias algo en la clase, aunque sea un cambio menor como renombrar un método o añadir un campo no relevante para la serialización, el compilador puede generar un nuevo serialVersionUID automáticamente, lo que puede romper la compatibilidad de la serialización entre diferentes versiones de la clase, generando una excepción **InvalidClassException**.

¿Cuándo debería cambiar el serialVersionUID? Solo cuando realices cambios en la clase que rompen la compatibilidad con versiones anteriores, por ejemplo: cambiar la jerarquía de herencia, eliminar atributos importantes, modificar el comportamiento de los datos de forma significativa, etc.

```
public class Coche implements Serializable {  
  
    private static final long serialVersionUID = -6144862803653714113L;  
}
```



UT2

ARCHIVOS DE ACCESO ALEATORIO

5. ARCHIVOS DE ACCESO ALEATORIO

INTRODUCCIÓN

Para trabajar con archivos de acceso aleatorio es **obligatorio conocer la estructura de los datos almacenados** y el tamaño de cada uno de los registros. Sin este conocimiento, podrías mover el puntero y leer datos, pero interpretarlos correctamente sería un desafío. La planificación y el diseño de cómo se estructuran los datos en un archivo son cruciales para el acceso aleatorio eficaz. Recuerda que su contenido se almacena como un **conjunto de bytes**.

Por este motivo, es importante tener siempre presente el número de bits/bytes que ocupa cada tipo de variable:

DATA TYPES	SIZE	DEFAULT	EXPLANATION
boolean	1 bit	false	Stores true or false values
byte	1 byte/ 8bits	0	Stores whole numbers from -128 to 127
short	2 bytes/ 16bits	0	Stores whole numbers from -32,768 to 32,767
int	4 bytes/ 32bits	0	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes/ 64bits	0L	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes/ 32bits	0.0f	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes/ 64bits	0.0d	Stores fractional numbers. Sufficient for storing 15 decimal digits
char	2 bytes/ 16bits	'\u0000'	Stores a single character/letter or ASCII values

5. ARCHIVOS DE ACCESO ALEATORIO

INTRODUCCIÓN

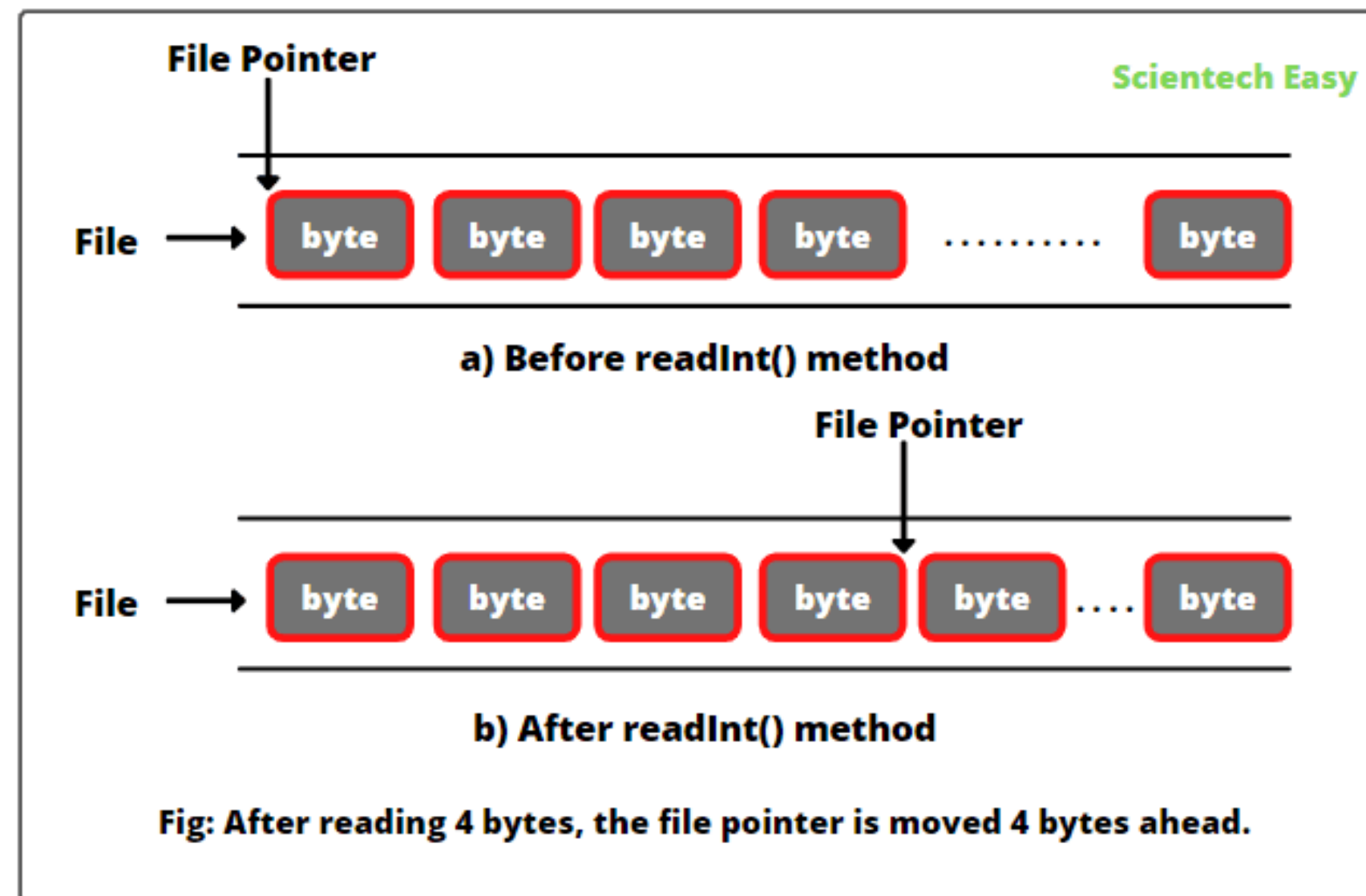
Para trabajar con los ficheros de acceso aleatorio utilizaremos la clase `RandomAccessFile`, cuyos métodos más importantes son:

- **`RandomAccessFile(String name, String mode)`**: constructor, crea un objeto `RandomAccessFile` para el archivo con el nombre dado en el modo especificado ("r" para lectura y "rw" para lectura y escritura).
- **`int readInt()`**: lee un entero de 4 bytes del archivo y devuelve su valor. También existen los métodos `readChar()`, `readDouble()`, etc.
- **`void writeInt()`**: escribe un entero en el archivo en 4 bytes. Al igual que con la lectura, existen los métodos `writeFloat()`, etc.
- **`long getFilePointer()`**: devuelve la posición actual del puntero en el archivo.
- **`void seek(long pos)`**: mueve el puntero a la posición especificada en el archivo.
- **`long length()`**: devuelve la longitud del archivo en bytes.
- **`void close()`**: cierra el archivo y libera los recursos asociados.

5. ARCHIVOS DE ACCESO ALEATORIO

INTRODUCCIÓN

El **puntero** en un RandomAccessFile se refiere a la posición actual en el archivo donde se pueden leer o escribir datos. Este puntero es crucial para el funcionamiento de RandomAccessFile, ya que permite acceder a diferentes partes del archivo de forma aleatoria, en lugar de tener que leer o escribir los datos en un orden secuencial.



5. ARCHIVOS DE ACCESO ALEATORIO

DEMO I - LECTURA Y ESCRITURA

En el siguiente ejemplo creamos un fichero de acceso aleatorio en el que insertamos 10 enteros:

```
public static void main(String[] args) {  
    try {  
        RandomAccessFile raf = new RandomAccessFile(name: "resources/numeros.dat", mode: "rw");  
  
        // Rellenamos el fichero  
        Random random = new Random();  
        for (int i = 0; i < 10; i++) {  
            raf.writeInt(random.nextInt());  
        }  
    } catch (FileNotFoundException e) {  
        throw new RuntimeException(e);  
    } catch (IOException e) {  
        throw new RuntimeException(e);  
    }  
}
```

* Ten en cuenta que `new RandomAccessFile()` no crea el fichero, debemos crearlo nosotros mediante la clase `File` como vimos al principio del tema

5. ARCHIVOS DE ACCESO ALEATORIO

DEMO I - LECTURA Y ESCRITURA

Sobre ese mismo fichero, vamos a leer todos los números. Para ello necesitamos un bucle que vaya desde la posición 0 hasta la longitud del fichero y hacer uso del método `readInt()`.

```
public static void leerFichero() {  
    try {  
        // Iniciamos solo en modo lectura  
        RandomAccessFile raf = new RandomAccessFile("resources/numeros.dat", "r");  
  
        // Recorremos el fichero hasta que el puntero llegue al final  
        while (raf.getFilePointer() < raf.length()) {  
            // Leemos el entero y lo mostramos  
            int numero = raf.readInt();  
            System.out.print(numero + ", ");  
        }  
        System.out.println(" ");  
  
    } catch (FileNotFoundException e) {  
        System.out.println("El fichero no existe.");  
        throw new RuntimeException(e);  
    } catch (IOException e) {  
        System.out.println("No se puede leer el contenido del fichero.");  
        throw new RuntimeException(e);  
    }  
}
```

5. ARCHIVOS DE ACCESO ALEATORIO

DEMO I - LECTURA Y ESCRITURA

También podemos leer una posición concreta del fichero, siempre y cuando exista:

```
public static void leerFicheroPosicion(int posicion) {
    try {
        // Iniciamos solo en modo lectura
        RandomAccessFile raf = new RandomAccessFile("resources/numeros.dat", "r");

        int cantidadNumeros = (int) raf.length() / Integer.BYTES;
        if (cantidadNumeros < posicion) {
            System.out.println("La posición indicada no existe.");
            return;
        }

        // Avanzamos el puntero a posicion * 4 (tamaño de cada byte)
        // Restamos 1 porque el primer entero se encuentra en la posición 4
        int posPuntero = (posicion - 1) * Integer.BYTES;
        raf.seek(posPuntero);

        System.out.println("El número que hay en la posición " + posicion
            + " es " + raf.readInt());
    } catch (FileNotFoundException e) {
        throw new RuntimeException(e);
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

5. ARCHIVOS DE ACCESO ALEATORIO

DEMO I - LECTURA Y ESCRITURA

```
RandomAccessFile raf = new RandomAccessFile(name: "resources/numeros.dat", mode: "rw");
```

```
// Insertamos los primeros 10 números naturales
```

```
for (int i = 0; i < 10; i++) {  
    raf.writeInt(i);  
}
```

```
// Un int ocupa 4 bytes por lo que cada elemento estará en  
// una posición múltiplo de 4: 0, 4, 8, etc.
```

```
// Vamos a suponer que intentamos leer el 6º entero (4 * 5 = 20)  
// pero nos hemos confundido y hemos leído desde el byte 19
```

```
raf.seek(pos: 19);
```

```
// 0 - 3   (num = 0)  00000000 00000000 00000000 00000000  
// 4 - 7   (num = 1)  00000000 00000000 00000000 00000001  
// 8 - 11  (num = 2)  00000000 00000000 00000000 00000010  
// 12 - 15 (num = 3)  00000000 00000000 00000000 00000011  
// 16 - 19 (num = 4)  00000000 00000000 00000000 00000100  
// 20 - 23 (num = 5)  00000000 00000000 00000000 00000101  
// 24 - 27 (num = 6)  00000000 00000000 00000000 00000110  
// .....
```

```
int sextoNumero = raf.readInt(); // 19 - 22: 00000100 00000000 00000000 00000000
```

```
System.out.println("El sexto número es: " + sextoNumero);
```

```
raf.close();
```

¿Qué pasa si me equivoco en el número de bytes que tengo que avanzar el puntero?

En ese caso vamos a leer contenido basura ya que parte de los bytes leídos serán de un elemento y otra parte serán del siguiente

El sexto número es: 67108864

Process finished with exit code 0

|

5. ARCHIVOS DE ACCESO ALEATORIO

DEMO II - LECTURA Y ESCRITURA DE CADENAS

¿Qué pasa si tenemos que almacenar un String?

Un String es esencialmente un array de caracteres de longitud variable. Esto significa que **su tamaño depende de la información almacenada**, lo cual complica su almacenamiento y procesamiento en archivos de acceso aleatorio, en los que debemos conocer el tamaño exacto de cada registro.

La solución a este problema es utilizar la clase *StringBuffer*, que permite asignar a todos los String una longitud fija, independientemente del contenido. Con el método `.setLength(int tamaño)`, podemos especificar el número exacto de caracteres que tendrá la cadena. Por ejemplo, si utilizamos `.setLength(10)`:

- Si el StringBuffer contiene el valor "Fernández" (9 caracteres), el método mantendrá esos 9 caracteres y añadirá un carácter nulo (\0) al final para alcanzar la longitud de 10 caracteres.
- Si el contenido es "transformador" (12 caracteres), el método truncará la palabra, conservando solo los primeros 10 caracteres.

Por eso, es crucial seleccionar adecuadamente el tamaño del buffer: demasiado pequeño puede truncar información importante, mientras que un tamaño excesivo consume memoria innecesariamente.

5. ARCHIVOS DE ACCESO ALEATORIO

DEMO II - LECTURA Y ESCRITURA DE CADENAS

**Ejemplo de inserción
de dos cadenas de
longitud fija 10:**

```
try {
    RandomAccessFile raf = new RandomAccessFile(new File(RUTA_FICHERO), mode: "rw");
    String apellido = "Gil";

    // Creamos el buffer. De momento tiene la misma longitud que el contenido del String
    StringBuffer buffer = new StringBuffer(apellido);

    // Ajustamos la longitud a nuestra necesidad
    buffer.setLength(10);

    // Almacenamos el buffer como un String, sabiendo que su tamaño son 10 caracteres
    raf.writeChars(buffer.toString());

    // Insertamos otro valor
    buffer = new StringBuffer("Fernandez");
    buffer.setLength(10);
    raf.writeChars(buffer.toString());

} catch (FileNotFoundException e) {
    throw new RuntimeException(e);
} catch (IOException e) {
    throw new RuntimeException(e);
}
```

5. ARCHIVOS DE ACCESO ALEATORIO

DEMO II - LECTURA Y ESCRITURA DE CADENAS

Ejemplo de lectura de un fichero que contiene cadenas de longitud fija 10:

```
try {
    RandomAccessFile raf = new RandomAccessFile(new File(RUTA_FICHERO), mode: "rw");

    // Leemos para comprobar que los datos introducidos son correctos
    raf.seek(pos: 0);
    while (raf.getFilePointer() < raf.length()) { // Leemos todas las cadenas del fichero
        String cadena = "";
        for (int i = 0; i < 10; i++) { // Cada cadena tiene 10 caracteres
            cadena = cadena + raf.readChar();
        }
        // Con trim eliminamos los espacios que haya
        System.out.println("Cadena: " + cadena.trim());
    }
} catch (FileNotFoundException e) {
    throw new RuntimeException(e);
} catch (IOException e) {
    throw new RuntimeException(e);
}
```

5. ARCHIVOS DE ACCESO ALEATORIO

DEMO III - LECTURA Y ESCRITURA DE OBJETOS COMPLEJOS

Demo en clase
a partir del
proyecto
DemoRAFDatos
Complejos

Vamos a suponer que queremos almacenar en un fichero de acceso aleatorio la información de los empleados de una empresa, de tal forma que cada registro va a contener la siguiente información:

empleado 1				empleado 2 ...	
entero (4bytes)	cadena (máx. 10 caracteres - 20 bytes)	entero (4bytes)	double (8bytes)	entero (4 bytes)	...
id	apellido	departamento	salario	id	...

Debemos tener claro que **cada empleado ocupa 36 bytes**, por lo que el empleado 1 va del byte 0 al 35, el empleado 2 del byte 36 al 71, etc.

¿A qué posición tendré que mover el puntero si quiero recuperar el departamento del empleado 4?

- Tengo que saltar los 3 primeros empleados: $36 \text{ bytes} * 3 \text{ empleados} = 108 \text{ bytes}$
- Tenemos que saltar el id y apellido del empleado 4: $4 \text{ bytes (id)} + 20 \text{ bytes (apellido)} = 24 \text{ bytes}$
- Posición final del puntero: $108 + 24 = 132 \text{ bytes}$